

API Security Risk Analysis Report

Introduction

Application Programming Interfaces (APIs) are an essential component of modern software applications. They enable communication between web applications, mobile applications, cloud services, and third-party systems. As organizations increasingly rely on APIs to exchange data and provide services, ensuring API security has become a critical requirement.

Insecure APIs can lead to unauthorized access, excessive data exposure, broken access controls, and other security vulnerabilities that may impact business operations and user privacy. Therefore, organizations regularly perform API security assessments to identify potential risks and strengthen their security posture.

This project focuses on conducting an API Security Risk Analysis of the JSONPlaceholder public API using Postman. The assessment was performed through documentation review, endpoint testing, response inspection, and security analysis while following ethical and read-only testing practices.

Objectives

The primary objectives of this API Security Risk Analysis are:

1. To analyze publicly accessible API endpoints using Postman.
2. To evaluate authentication and authorization mechanisms.
3. To identify common API security risks and weaknesses.
4. To inspect API responses for excessive data exposure.
5. To review HTTP security headers and rate-limiting controls.
6. To assess the potential business impact of identified risks.
7. To provide practical remediation recommendations.
8. To understand real-world API security assessment methodologies used by security analysts and consultants.

Scope

The scope of this project is limited to the security assessment of the JSONPlaceholder public API using safe and ethical testing methods. The analysis focuses on reviewing publicly accessible API endpoints, inspecting request and response data, evaluating authentication and authorization mechanisms, examining security headers, and identifying common API security risks.

The assessment was conducted using Postman and browser-based testing through read-only requests without performing any exploitation, denial-of-service attacks, or unauthorized access attempts. The endpoints included in the analysis are **/posts**, **/users**, and **/comments**.

This project aims to identify potential security weaknesses, assess their business impact, and recommend appropriate remediation measures based on industry best practices and OWASP API Security guidelines.

Tools Used

The following tools were used during the API Security Risk Analysis:

1. Postman

Postman was used to send API requests, inspect responses, analyze headers, and evaluate endpoint behavior.

2. Web Browser

A web browser was used to access API endpoints directly and verify response data for authorization analysis.

3. GitHub

GitHub was used to maintain project documentation, store screenshots, and publish the final project repository.

4. JSONPlaceholder API

JSONPlaceholder served as the public test API used for conducting the security assessment and analyzing potential security risks.

5. API Documentation

The official API documentation was reviewed to understand endpoint functionality, request methods, and available resources.

Methodology

The API Security Risk Analysis was conducted using a structured and ethical assessment approach. The methodology focused on identifying common API security risks through observation, documentation review, and read-only testing.

Step 1: API Selection

A public test API (JSONPlaceholder) was selected for analysis because it is designed for learning and testing purposes.

Step 2: Documentation Review

The API documentation was reviewed to understand available endpoints, request methods, and response structures.

Step 3: Endpoint Testing

API endpoints such as **/posts**, **/users**, and **/comments** were tested using Postman to observe their behavior and responses.

Step 4: Authentication Analysis

The API was examined to determine whether authentication mechanisms such as API keys, tokens, or user login were required to access resources.

Step 5: Authorization Review

Different resource identifiers were tested to evaluate whether access control mechanisms were implemented correctly.

Step 6: Response Inspection

API responses were analyzed to identify excessive data exposure and unnecessary disclosure of user information.

Step 7: Header Analysis

HTTP response headers were inspected to check for the presence of important security headers.

Step 8: Risk Assessment

The identified issues were classified according to their potential severity and business impact.

Step 9: Remediation Recommendations

Appropriate security recommendations were provided based on industry best practices and OWASP API Security guidelines.

API Tested

API Name

JSONPlaceholder – Fake Online REST API for Testing and Prototyping

Base URL

<https://jsonplaceholder.typicode.com>

Purpose

JSONPlaceholder is a free public API designed for developers, testers, and students to practice API testing and application development. It provides sample data and supports common REST API operations.

Endpoints Tested

1. Posts Endpoint

URL: <https://jsonplaceholder.typicode.com/posts>

Method: GET

Purpose: Retrieve a list of posts and analyze response behavior.

2. Users Endpoint

URL: <https://jsonplaceholder.typicode.com/users>

Method: GET

Purpose: Review user-related information and identify potential data exposure risks.

3. Comments Endpoint

URL: <https://jsonplaceholder.typicode.com/comments>

Method: GET

Purpose: Examine response structure and assess publicly accessible data.

Testing Environment

- Tool Used: Postman
- Testing Type: Read-Only Assessment
- Assessment Focus:
 - Authentication Analysis
 - Authorization Review
 - Data Exposure Analysis

- Security Header Inspection
- Rate Limiting Review

Findings & Risk Analysis

Finding 1: Unauthenticated Access

Severity

Medium

Observation

The tested API endpoints were accessible without requiring authentication credentials such as API keys, access tokens, or user login.

Risk Analysis

The absence of authentication controls allows any user to access publicly available API resources. In a production environment, this could lead to unauthorized access to sensitive information and increased security risks.

Business Impact

Unauthorized users may gain access to application data, potentially resulting in privacy concerns and information disclosure.

Recommendation

Implement secure authentication mechanisms such as JWT, OAuth 2.0, or API key-based authentication.

Finding 2: Excessive Data Exposure

Severity

Medium

Observation

The `/users` endpoint returns multiple user-related fields including name, email, phone number, address, website, and company information.

Risk Analysis

Providing more information than necessary increases the attack surface and may expose data that should not be publicly accessible.

Business Impact

Excessive data exposure may lead to privacy issues, information leakage, and regulatory compliance concerns.

Recommendation

Limit API responses to only the fields required by the client application.

Finding 3: Broken Object Level Authorization (BOLA)

Severity

Medium

Observation

Different user records can be accessed by modifying the user identifier in the endpoint URL.

Example:

- `/users/1`
- `/users/2`
- `/users/3`

Risk Analysis

If proper authorization checks are not implemented, users may access resources belonging to other users.

Business Impact

Unauthorized access to user data may result in confidentiality breaches and loss of trust.

Recommendation

Validate user ownership and permissions before returning resources.

Finding 4: Missing Security Headers

Severity

Low

Observation

Important security headers such as Content-Security-Policy (CSP), X-Frame-Options, and Strict-Transport-Security (HSTS) were not clearly visible during analysis.

Risk Analysis

Missing security headers may increase the likelihood of browser-based attacks and weaken overall security posture.

Business Impact

Applications may become more vulnerable to clickjacking, content injection, and other web-based threats.

Recommendation

Configure and enforce recommended HTTP security headers.

Finding 5: Missing Rate Limiting

Severity

Low

Observation

No visible rate-limiting mechanism was identified during testing.

Risk Analysis

Without request throttling, APIs may be more susceptible to automated abuse, excessive requests, and scraping activities.

Business Impact

High volumes of requests may affect system availability and performance.

Recommendation

Implement rate limiting and request throttling controls to restrict excessive API usage.

Risk Assessment Summary

The identified risks were evaluated based on their potential impact on security, confidentiality, and API misuse.

S.No	Risk Identified	Severity	Business Impact
1	Unauthenticated Access	Medium	Unauthorized users may access API resources without authentication.
2	Excessive Data Exposure	Medium	Unnecessary user information may be disclosed, increasing privacy risks.
3	Broken Object Level Authorization (BOLA)	Medium	Users may gain access to resources belonging to other users.
4	Missing Security Headers	Low	Increases exposure to browser-based attacks and security weaknesses.

5	Missing Rate Limiting	Low	May allow excessive requests, automated abuse, and data scraping.
---	-----------------------	-----	---

Overall Risk Rating

Based on the assessment, the overall security risk level is classified as **Medium**. The API demonstrates several common security concerns related to authentication, authorization, and information exposure. While the identified issues do not indicate immediate critical vulnerabilities in this test environment, implementing recommended security controls would significantly improve the API's security posture and reduce potential risks.

Remediation Recommendations

Based on the findings identified during the API Security Risk Analysis, the following remediation measures are recommended to strengthen the security of the API and reduce potential risks.

1. Implement Strong Authentication

Recommendation

Use secure authentication mechanisms such as JWT (JSON Web Tokens), OAuth 2.0, or API Keys to verify the identity of users before granting access to API resources.

Benefit

Prevents unauthorized users from accessing sensitive API endpoints and data.

2. Enforce Proper Authorization Controls

Recommendation

Implement role-based access control (RBAC) and validate user permissions before returning resources. Ensure that users can only access data that belongs to them.

Benefit

Reduces the risk of Broken Object Level Authorization (BOLA) vulnerabilities and unauthorized data access.

3. Minimize Data Exposure

Recommendation

Return only the information required by the client application. Avoid exposing unnecessary fields such as personal details, contact information, or internal system data.

Benefit

Protects user privacy and reduces the risk of information disclosure.

4. Enable Security Headers

Recommendation

Configure important HTTP security headers including:

- Content-Security-Policy (CSP)
- X-Frame-Options
- Strict-Transport-Security (HSTS)
- X-Content-Type-Options

Benefit

Provides additional protection against browser-based attacks such as clickjacking and content injection.

5. Implement Rate Limiting

Recommendation

Apply request throttling and rate-limiting mechanisms to control the number of requests that can be made within a specific time period.

Benefit

Protects the API from abuse, automated attacks, excessive requests, and scraping activities.

6. Perform Regular Security Assessments

Recommendation

Conduct periodic API security reviews, vulnerability assessments, and security testing to identify emerging risks.

Benefit

Helps maintain a strong security posture and ensures continuous improvement.

7. Follow OWASP API Security Best Practices

Recommendation

Align API development and security controls with the OWASP API Security Top 10 guidelines.

Benefit

Reduces exposure to common API vulnerabilities and improves overall security maturity.

Conclusion

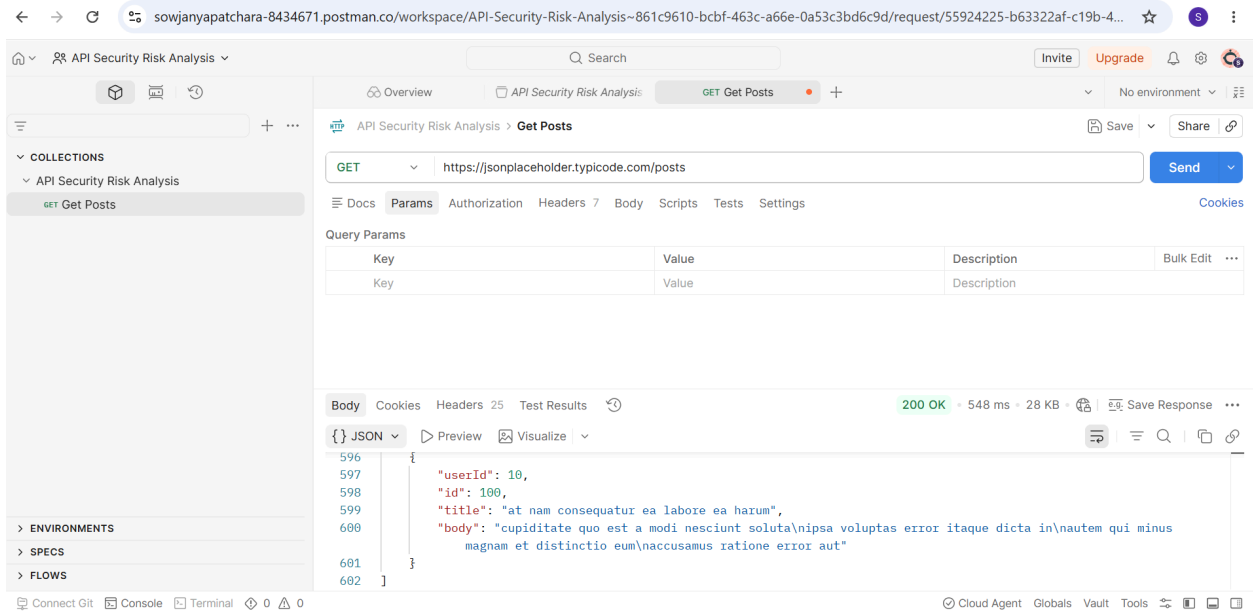
This API Security Risk Analysis was conducted to evaluate the security posture of the JSONPlaceholder public API using ethical and read-only testing techniques. The assessment focused on authentication, authorization, data exposure, security headers, and rate-limiting mechanisms.

During the analysis, several common API security concerns were identified, including unauthenticated access, excessive data exposure, Broken Object Level Authorization (BOLA), missing security headers, and the absence of visible rate-limiting controls. These findings highlight the importance of implementing strong security measures to protect APIs from unauthorized access, information disclosure, and potential misuse.

Although the assessed API is intended for testing and educational purposes, the methodology used in this project reflects real-world API security assessment practices followed by security analysts and consultants. By implementing the recommended remediation measures, organizations can significantly improve API security, reduce potential risks, and strengthen overall application security.

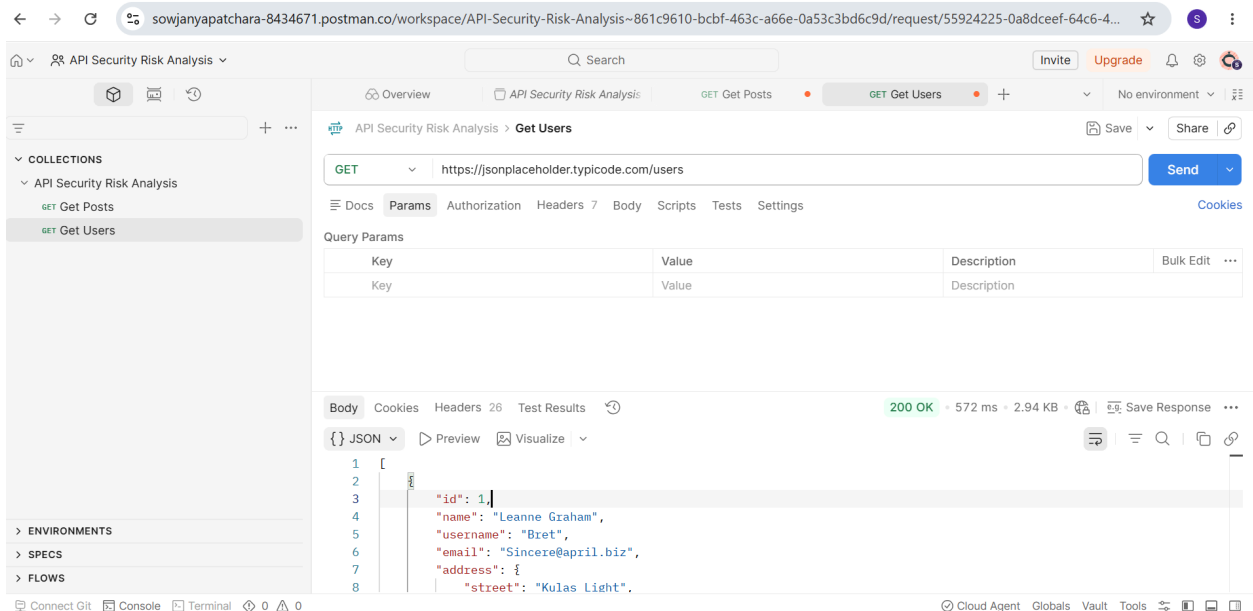
This project provided practical experience in API testing, risk identification, security analysis, and report preparation, while demonstrating the importance of secure API design and adherence to industry best practices such as the OWASP API Security Top 10.

Screenshots



The screenshot shows the Postman interface for a workspace named "API Security Risk Analysis". The active collection is "API Security Risk Analysis" and the selected item is "GET Get Posts". The request is a GET method to the URL "https://jsonplaceholder.typicode.com/posts". The response is a 200 OK status with a response time of 548 ms and a body size of 28 KB. The response body is a JSON object representing a single post.

```
{
  "userId": 10,
  "id": 100,
  "title": "at nam consequatur ea labore ea harum",
  "body": "cupiditate quo est a modi nesciunt soluta nipsa voluptas error itaque dicta in nauteum qui minus magnam et distinctio eum naccusamus ratione error aut"
}
```



The screenshot shows the Postman interface for the same workspace. The active collection is "API Security Risk Analysis" and the selected item is "GET Get Users". The request is a GET method to the URL "https://jsonplaceholder.typicode.com/users". The response is a 200 OK status with a response time of 572 ms and a body size of 2.94 KB. The response body is a JSON array containing a single user object.

```
[
  {
    "id": 1,
    "name": "Leanne Graham",
    "username": "Bret",
    "email": "Sincere@april.biz",
    "address": {
      "street": "Kulas Light",

```

← → ↻ sowjanyapatchara-8434671.postman.co/workspace/API-Security-Risk-Analysis~861c9610-bcbf-463c-a66e-0a53c3bd6c9d/request/55924225-4e96343d-b06e-4... ☆ ⓘ

API Security Risk Analysis

Overview API Security Risk Anal GET Get Posts GET Get Users GET Get Comments + No environment Cookies

API Security Risk Analysis > Get Comments

GET https://jsonplaceholder.typicode.com/comments Send

Docs Params Authorization Headers 7 Body Scripts Tests Settings

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 25 Test Results 200 OK • 369 ms • 155.17 KB Save Response

{ } JSON Preview Visualize

```
1 [
2   {
3     "postId": 1,
4     "id": 1,
5     "name": "id labore ex et quam laborum",
6     "email": "Eliseo@gardner.biz",
7     "body": "laudantium enim quasi est quidem magnam voluptate ipsam eos\ntempora quo necessitatibus\ndolor quam autem quasi\nreiciendis et nam sapiente accusantium"
  }
]
```

Connect Git Console Terminal 0 Cloud Agent Globals Vault Tools

← → ↻ sowjanyapatchara-8434671.postman.co/workspace/API-Security-Risk-Analysis~861c9610-bcbf-463c-a66e-0a53c3bd6c9d/request/55924225-0a8dceef-64c6-4... ☆ ⓘ

API Security Risk Analysis

Overview API Security Risk Anal GET Get Posts GET Get Users GET Get Comments + No environment Cookies

API Security Risk Analysis > Get Users

GET https://jsonplaceholder.typicode.com/users Send

Docs Params Authorization Headers 7 Body Scripts Tests Settings

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 26 Test Results 200 OK • 572 ms • 2.94 KB Save Response

Key	Value
Date	Mon, 15 Jun 2026 14:55:00 GMT
Content-Type	application/json; charset=utf-8
Content-Length	1847
Connection	keep-alive
access-control-allow-credentials	true

Connect Git Console Terminal 0 Cloud Agent Globals Vault Tools

← → ↻ jsonplaceholder.typicode.com/users/3



pretty print ☐

```
{
  "id": 3,
  "name": "Clementine Bauch",
  "username": "Samantha",
  "email": "Nathan@yesenia.net",
  "address": {
    "street": "Douglas Extension",
    "suite": "Suite 847",
    "city": "McKenziehaven",
    "zipcode": "59590-4157",
    "geo": {
      "lat": "-68.6102",
      "lng": "-47.0653"
    }
  },
  "phone": "1-463-123-4447",
  "website": "ramiro.info",
  "company": {
    "name": "Romaguera-Jacobson",
    "catchPhrase": "Face to face bifurcated interface",
    "bs": "e-enable strategic applications"
  }
}
```